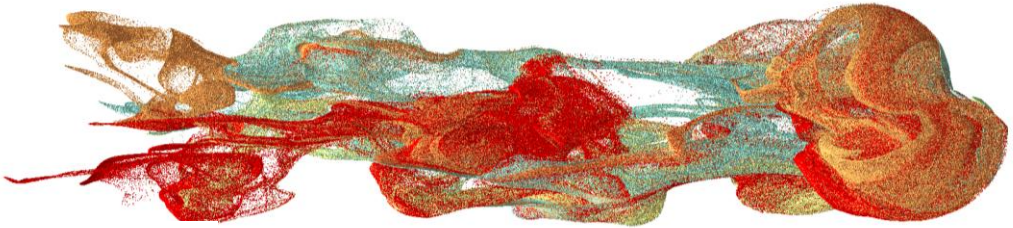
Simulación de fluidos: “Stable Fluids”

Real-Time Fluid Dynamics for Games



U-tad CENTRO UNIVERSITARIO DE TECNOLOGÍA Y ARTE DIGITAL

Diego Rojo

Simulación de fluidos

- Un fluido es una sustancia que carece de forma propia.

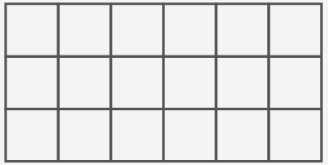
Simulación de fluidos

Métodos Eulerianos

- Las propiedades del fluido se registran en puntos que permanecen fijos mientras el fluido se mueve
- Enfoque más clásico, más fácil de hacer preciso en el límite, difícil de hacer rápido

Métodos Lagrangianos

- Las propiedades del fluido se registran en puntos que se mueven con el fluido
- Puede ser más simple, más fácil de hacer rápido, más difícil de lograr alta precisión



grid-based

Leonhard Euler
(1707–1783)

grid-free (e. g. particles)

Joseph-Louis Lagrange
(1736–1813)

Simulación de fluidos

Métodos Híbridos

- Convertir de ida y vuelta entre representaciones Eulerianas y Lagrangianas con la esperanza de obtener las ventajas de ambas

Stable Fluids by Jos Stam

Simulación de fluidos en tiempo real

- Presentación: “Stable Fluids” — Jos Stam (1999)
- Implementación: “Real-Time Fluid Dynamics for Games” — Jos Stam (2003)

STABLE FLUIDS

Jos Stam (1999)
SIGGRAPH 1999 Conference proceedings:
SIGGRAPH Annual Conference Series
pp. 121-128

All right so this is probably my most cited paper. It was however the paper with the lowest scores that got accepted that year at SIGGRAPH. That is a story I like to tell. It is actually very telling that impact papers usually get low scores and get rejected. I was lucky that some people believed in the paper on the committee. I first got this to work in 1997 when I just moved to Seattle and started to play with some code I wrote in Finland. The code solved for the motion of a density given a kinetic velocity field. Then I just boldly applied the same technique to the velocity field itself making it dynamic. It took no time to code and it worked right away. Real-time fluids! You can imagine my excitement. Research code usually doesn't work right away, certainly not the first time you run it. When research code doesn't work you are faced with a dilemma. Either it is a bug or your idea is flawed. This is not always easy to decide in practice. When you implement a known technique that you know works then you know it is a bug in your code. Well sometimes you might run into a flaw of the technique. The presentation at SIGGRAPH was rock and roll. I took a cab with an sgi Octane down to the convention center and we switched a defective graphics card the night before. But in the end it all worked out and I was able to show for the first time real-time fluid simulations in front of a large audience.

pdf slides

REAL-TIME FLUID DYNAMICS FOR GAMES

Jos Stam (2003)
GDC 2003 Conference proceedings:
Game Developers Conference

In 2002 my company asked me to submit something about fluids to the annual Gamer Conference. I had been coding on small devices and considerably simplified my code. I brought it down to roughly 100 lines of readable and self-contained C code. I included it in my GDC paper and on the accompanying CD. There is no more arguing whether Stable Fluids is easy to implement. Download and try it at home. Still some people complained I didn't include a Makefile. I wish most papers would release some sort of simple code of their technique to get people started.

pdf slides

Stable Fluids by Jos Stam

Simulación de fluidos en tiempo real

- Presentación: “Stable Fluids” — Jos Stam (1999)
- Implementación: “Real-Time Fluid Dynamics for Games” — Jos Stam (2003)
- Base de MAYA Fluid Effects ™

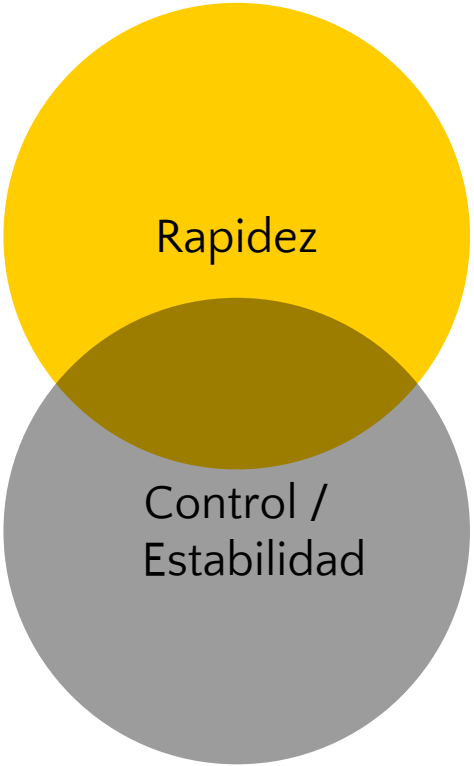
U-tad CENTRO UNIVERSITARIO DE TECNOLOGÍA Y ARTE DIGITAL

MAYA Fluid Effects

U-tad CENTRO UNIVERSITARIO DE TECNOLOGÍA Y ARTE DIGITAL

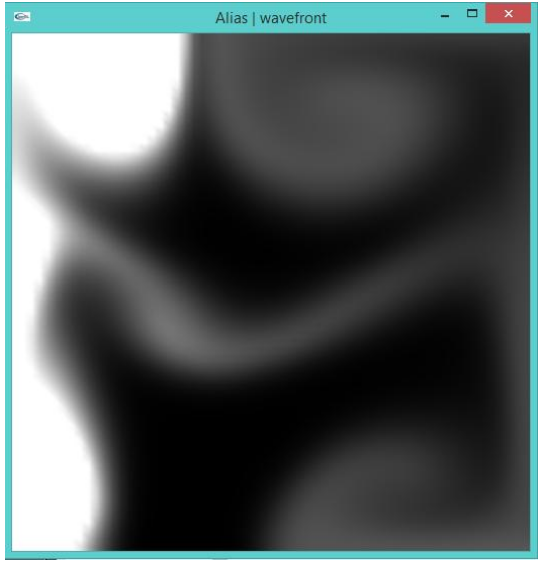
MAYA Fluid Effects

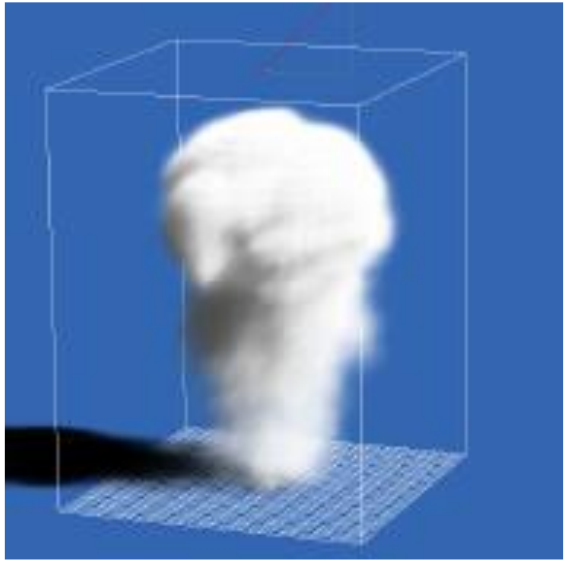
Objetivos



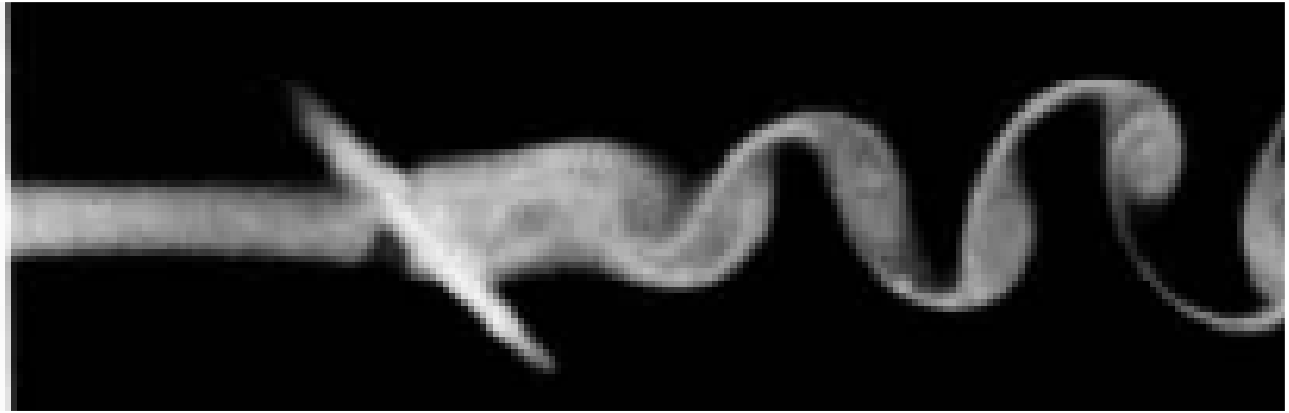
- No preciso

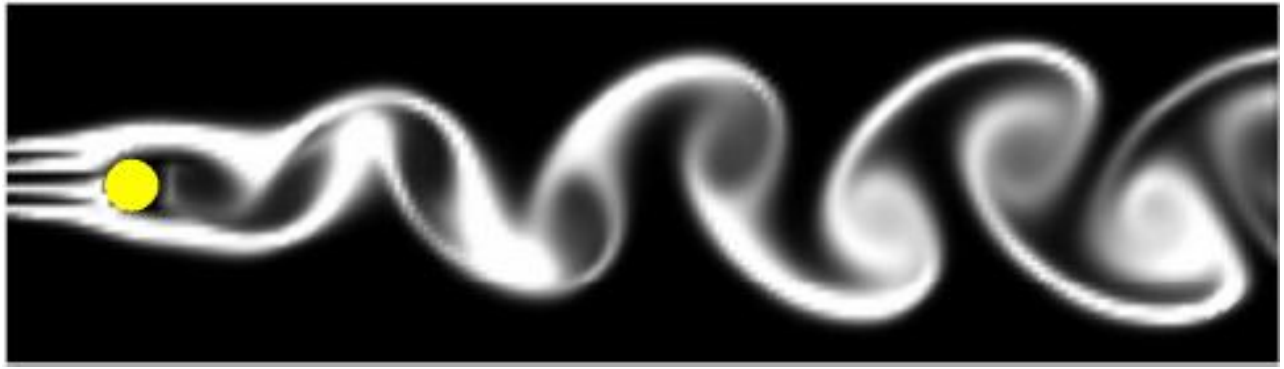
2D o 3D



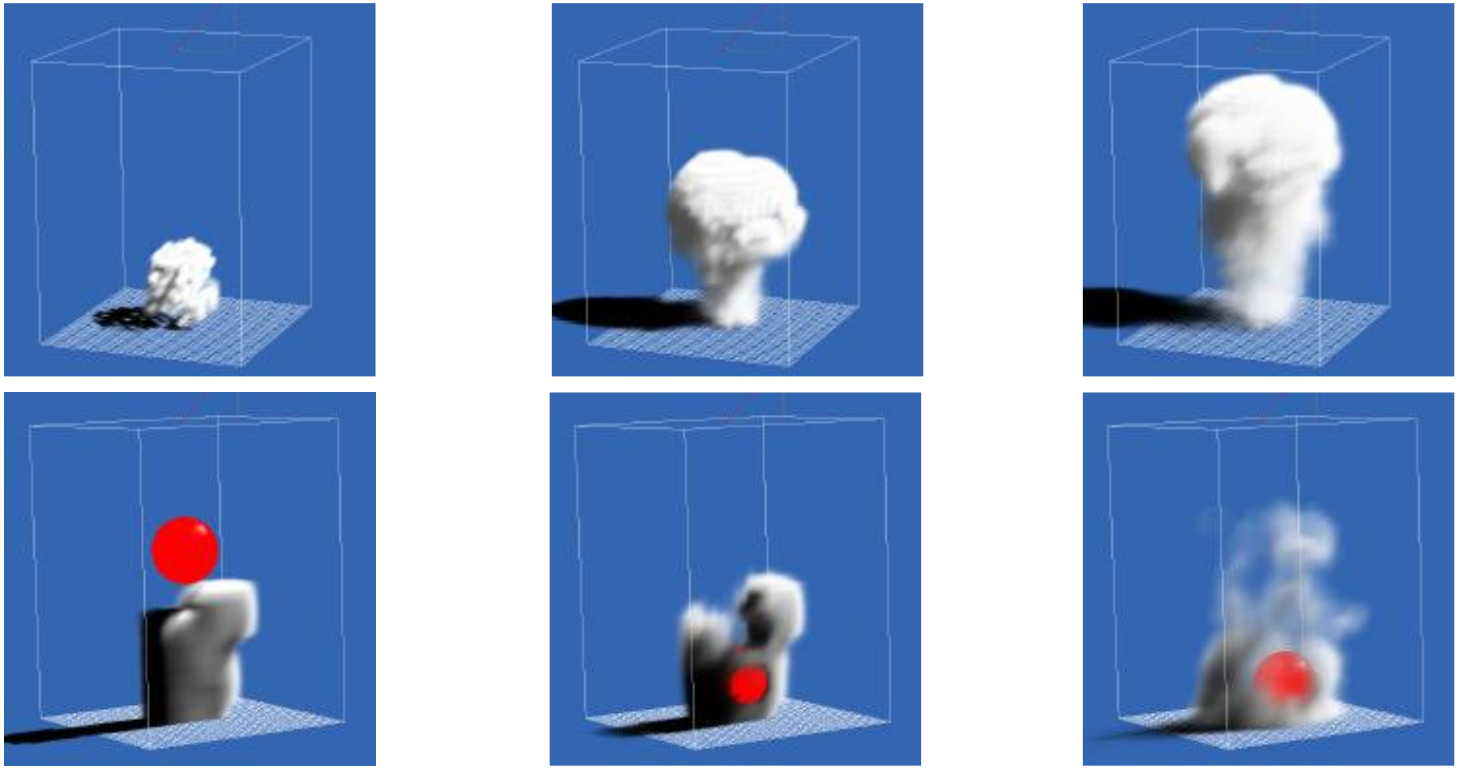


Validación: ¿Visualmente realista?

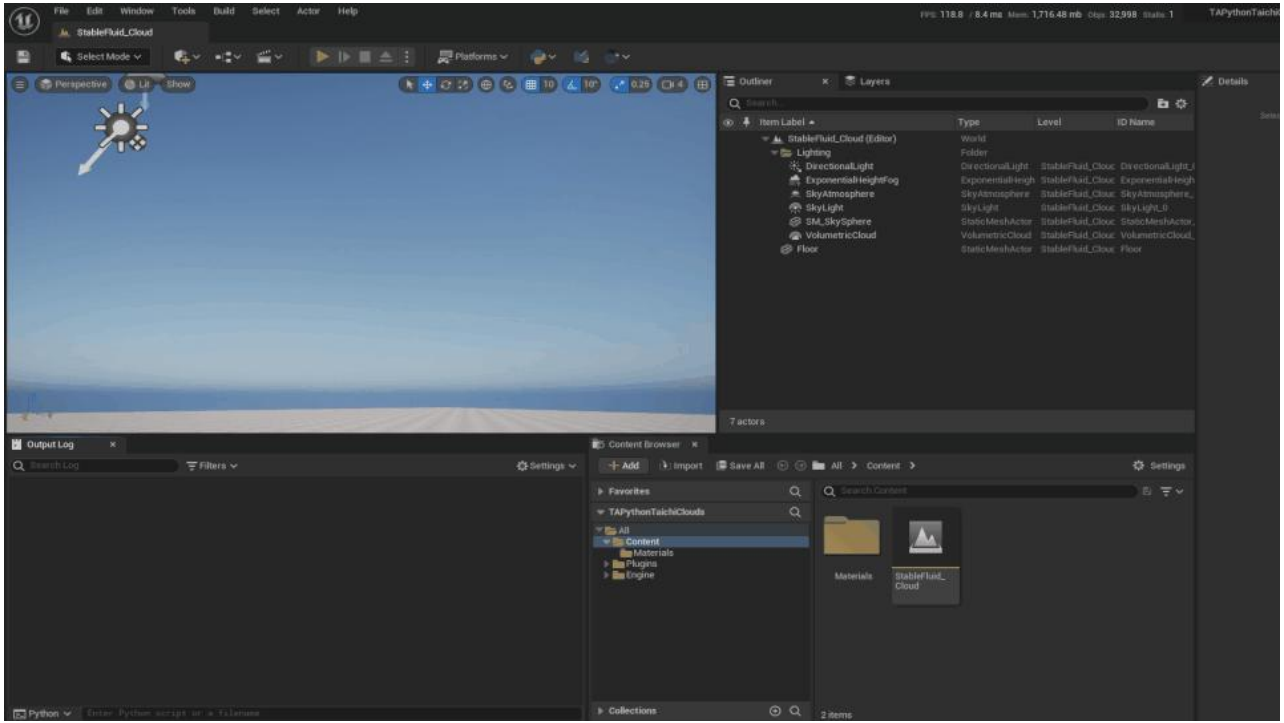




Validación: ¿Visualmente realista?



Validación: ¿Visualmente realista?



https://github.com/cgerchenhp/TAPython_Taichi_StableFluid_UE5

Diseño del modelo

Modelo preciso para la mayoría de movimientos de fluidos *incompresibles* que ocurren en la naturaleza:

- Ecuaciones de Navier-Stokes

$$
\begin{aligned}
\nabla \cdot \mathbf{u} &= 0 \\
\frac{\partial \mathbf{u}}{\partial t} &= -(\mathbf{u} \cdot \nabla)\mathbf{u} + \nu \nabla^2 \mathbf{u} + \mathbf{f}
\end{aligned}
$$

Diseño del modelo

Modelo preciso para la mayorí
de fluidos *incompresibles* que
naturaleza:

- Ecuaciones de Navier-Stokes

$$
\begin{aligned}
\nabla \cdot \mathbf{u} &= 0 \\
\frac{\partial \mathbf{u}}{\partial t} &= -(\mathbf{u} \cdot \nabla)\mathbf{u} + \nu \nabla^2 \mathbf{u} + \mathbf{f}
\end{aligned}
$$

Operator	Definition
Gradient	$\nabla p = \left(\frac{\partial p}{\partial x}, \frac{\partial p}{\partial y} \right)$
Divergence	$\nabla \cdot \mathbf{u} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$
Laplacian	$\nabla^2 p = \frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2}$

Diseño del modelo

Modelo preciso para la mayoría de movimientos de fluidos *incompresibles* que ocurren en la naturaleza:

- Ecuaciones de Navier-Stokes

$$
\begin{aligned}
\nabla \cdot \mathbf{u} &= 0 \\
\frac{\partial \mathbf{u}}{\partial t} &= -(\mathbf{u} \cdot \nabla)\mathbf{u} + \nu \nabla^2 \mathbf{u} + \mathbf{f}
\end{aligned}
$$

$\boldsymbol{u}$: velocidad del fluido
ρ: densidad del fluido
k: presión o fuerza que ejerce el fluido por unidad de superficie
v: viscosidad cinemática
f: fuerzas externas
S: fuentes

Modelo: ecuaciones

$$\frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla)\mathbf{u} + \nu \nabla^2 \mathbf{u} + \mathbf{f}$$

- Describe campo vectorial de velocidades de un fluido.
- Visualizar el fluido: densidad (e.g. humo/colorante)

$$\frac{\partial \rho}{\partial t} = -(\mathbf{u} \cdot \nabla)\rho + \kappa \nabla^2 \rho + S$$

Evolución de la densidad

- Supongamos que la velocidad es conocida.

$$\boxed{\frac{\partial \rho}{\partial t}} = -(\mathbf{u} \cdot \nabla)\rho + \kappa \nabla^2 \rho + S$$

En un paso de tiempo la densidad...

Evolución de la densidad

- Supongamos que la velocidad es conocida.

$$\frac{\partial \rho}{\partial t} = \boxed{-(\mathbf{u} \cdot \nabla)\rho} + \kappa \nabla^2 \rho + S$$

Sigue el vector de velocidades

Evolución de la densidad

- Supongamos que la velocidad es conocida.

$$\frac{\partial \rho}{\partial t} = -(\mathbf{u} \cdot \nabla)\rho + \boxed{\kappa \nabla^2 \rho} + S$$

Puede sufrir difusión con el tiempo

Evolución de la densidad

- Supongamos que la velocidad es conocida.

$$\frac{\partial \rho}{\partial t} = -(\mathbf{u} \cdot \nabla)\rho + \kappa \nabla^2 \rho + \boxed{S}$$

Incrementa debido a las fuentes que se añaden

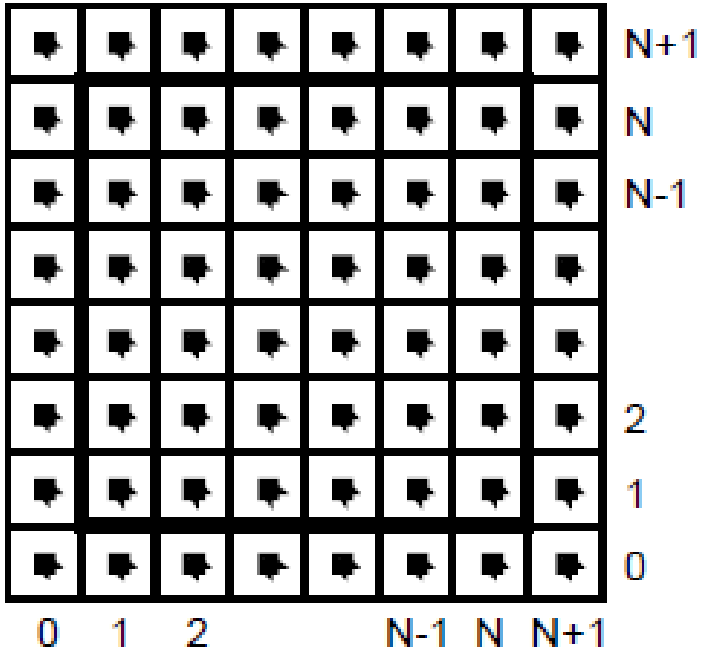
Implementación: pseudocódigo

While (simulando)

- Obtener fuentes de densidad del UI
- Obtener fuerza del UI (interfaz usuario)
- Actualizar velocidad
- Actualizar densidad
- Mostrar densidad

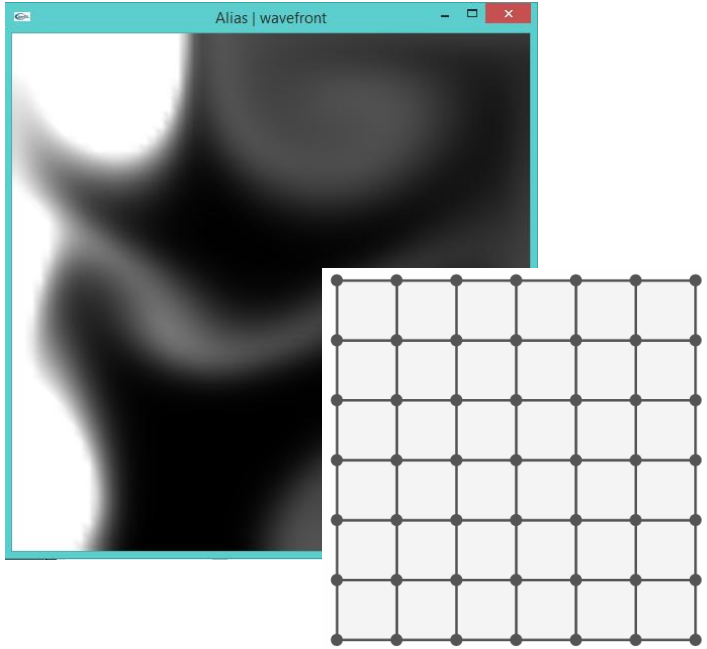
Algoritmo

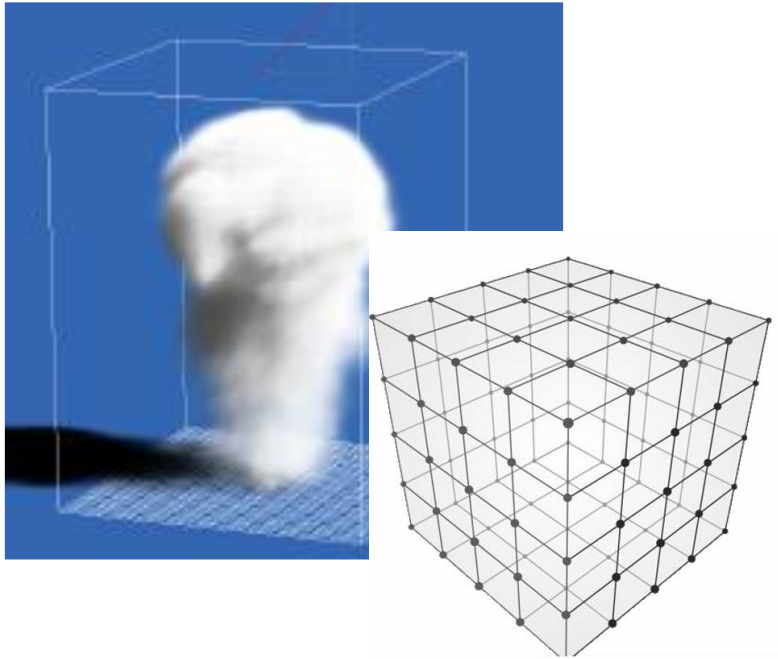
- 2D (razonamiento válido para 3D)



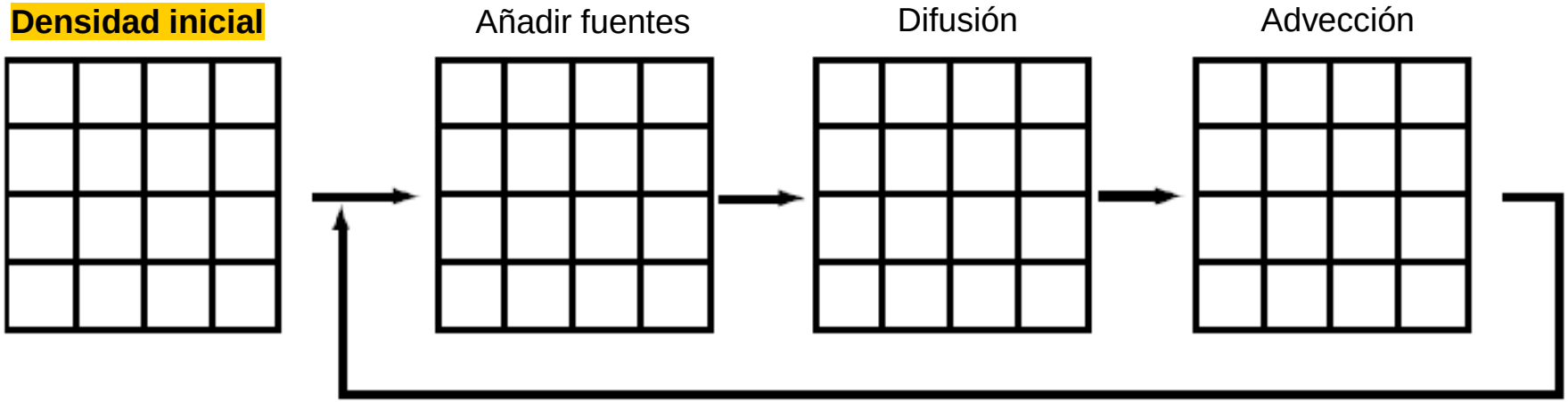
- Discretizamos
- Densidad y velocidad constante en cada celda

2D o 3D



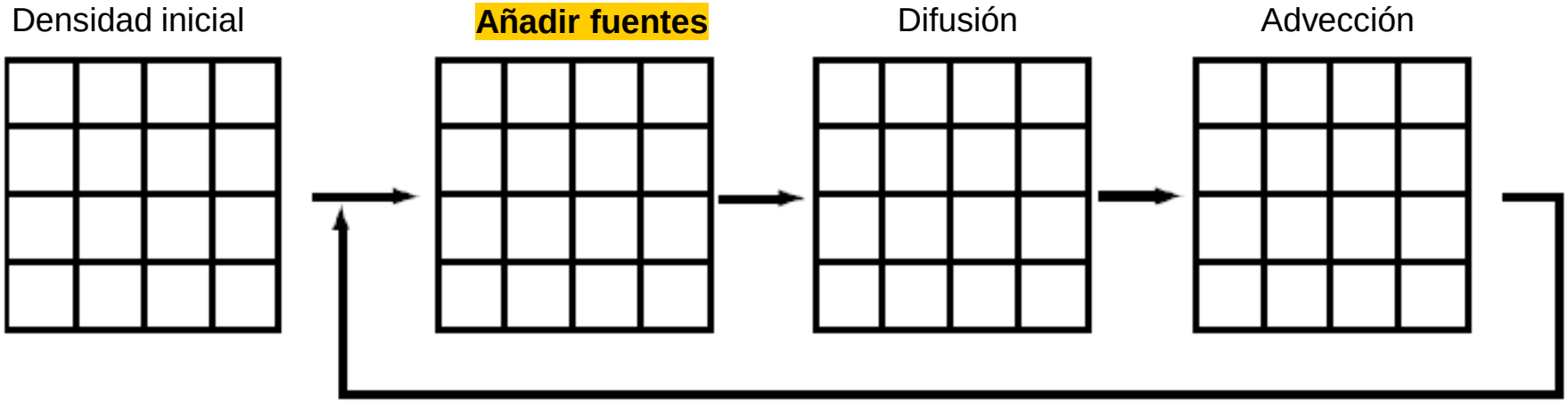


Algoritmo : densidades



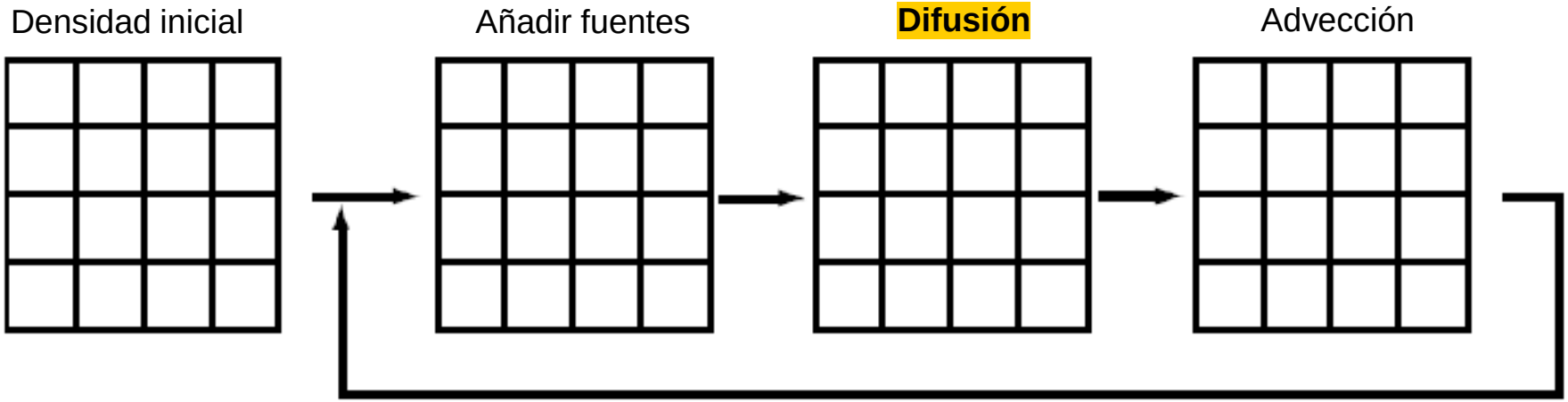
$$d_{ij}^{0}$$

Algoritmo : añadir fuentes

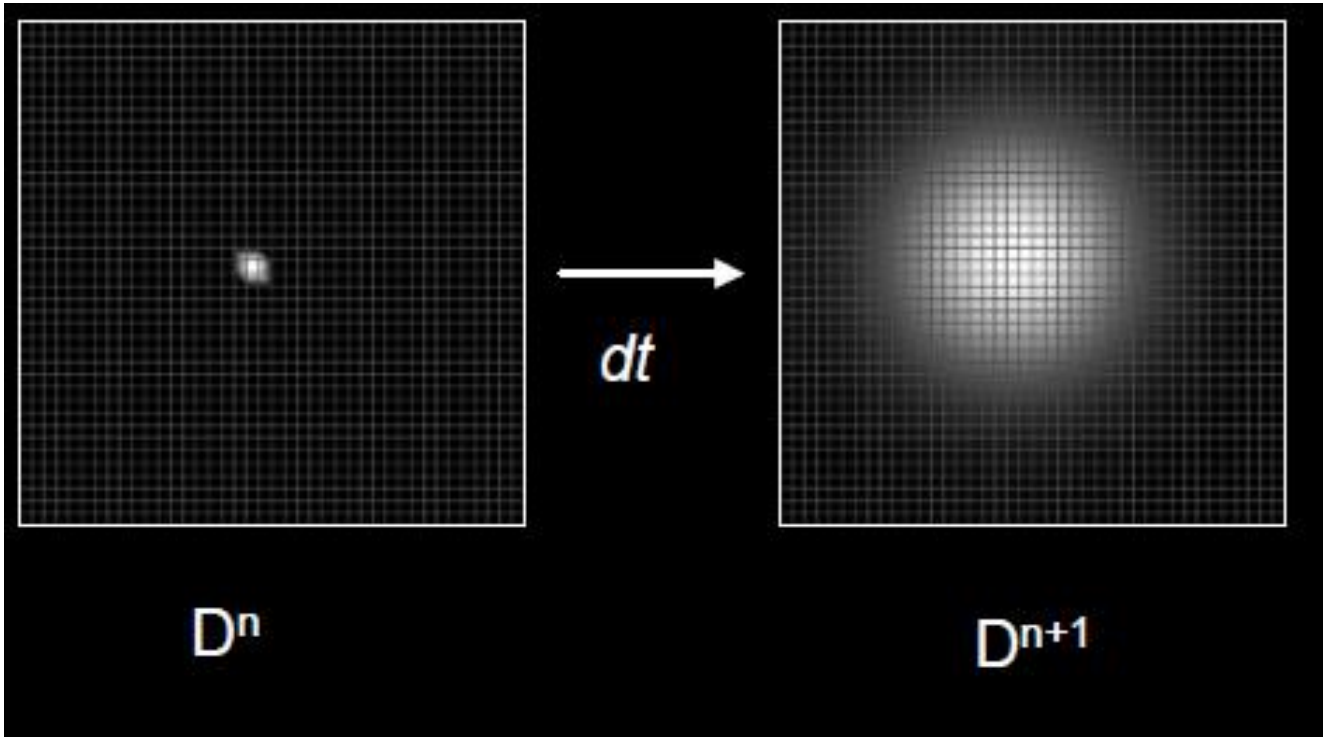


$$d_{ij} = d_{ij}^0 + \Delta t s_{ij}$$

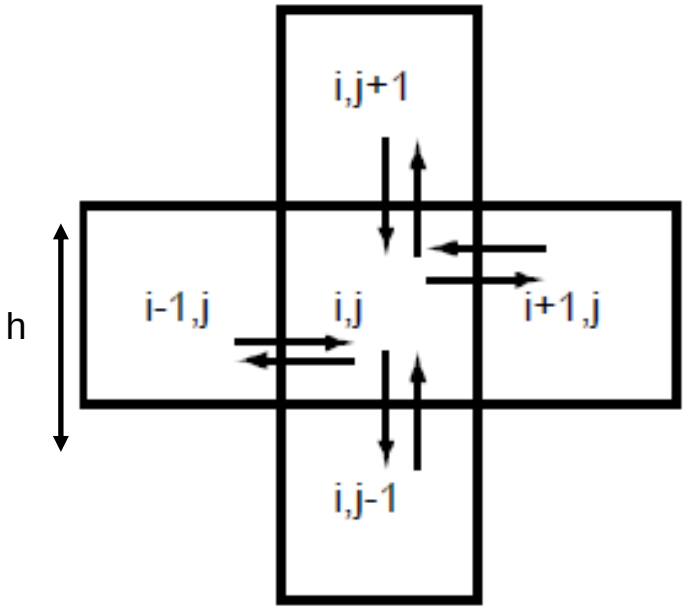
Algoritmo : difusión



Algoritmo : difusión



Algoritmo : difusión

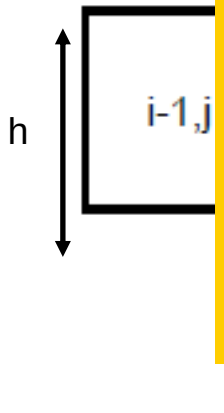


- Cada celda intercambia densidad con sus vecinas.
- h longitud de paso.
- k coeficiente de difusión.

$$d_{ij}^{n+1} = d_{ij}^{n} + \kappa \Delta t (d_{i-1,j}^{n} + d_{i+1,j}^{n} + d_{i,j-1}^{n} + d_{i,j+1}^{n} - 4d_{ij}^{n})/h^2$$

Algoritmo : difusión

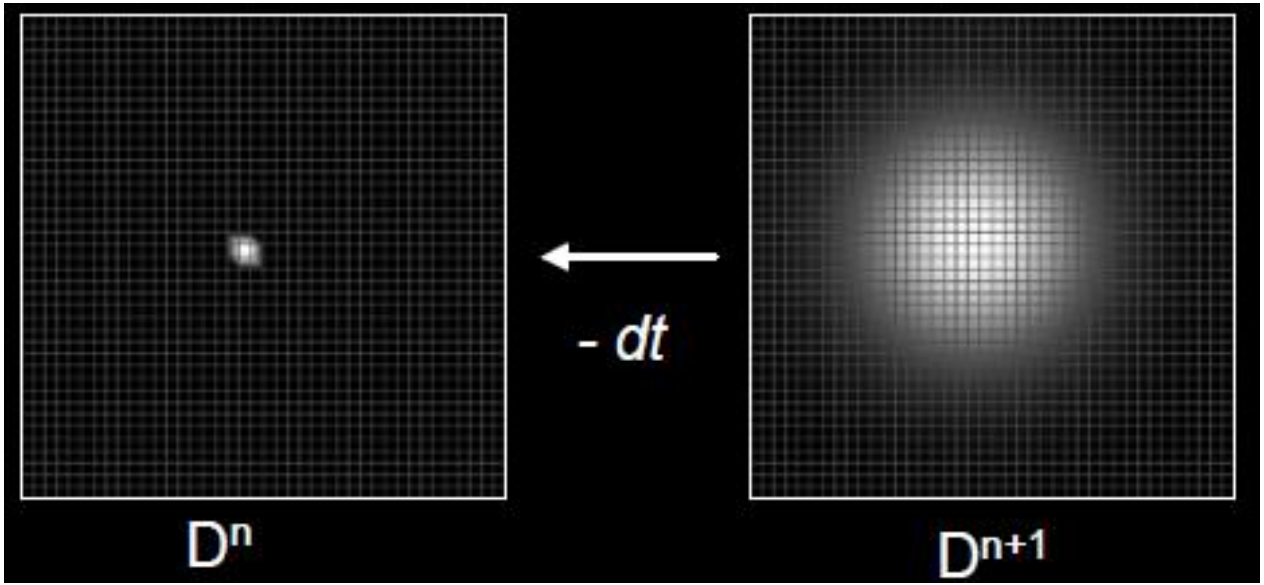
ercambia
vecinas.
paso.
e difusión.



Inestable si : $\kappa \Delta t / h^2 > \frac{1}{2}$

$$d_{ij}^{n+1} = d_{ij}^{n} + \kappa \Delta t (d_{i-1,j}^{n} + d_{i+1,j}^{n} + d_{i,j-1}^{n} + d_{i,j+1}^{n} - 4 d_{ij}^{n}) / h^2$$

Algoritmo : difusión



Buscamos densidades que sufriendo difusión “marcha atrás” nos den la densidades originales.

Algoritmo : difusión

- Sistema lineal (A*x=b con A dispersa) :

$$d_{ij}^{n} = d_{ij}^{n+1} - \kappa \Delta t (d_{i-1,j}^{n+1} + d_{i+1,j}^{n+1} + d_{i,j-1}^{n+1} + d_{i,j+1}^{n+1} - 4 d_{ij}^{n+1}) / h^2$$

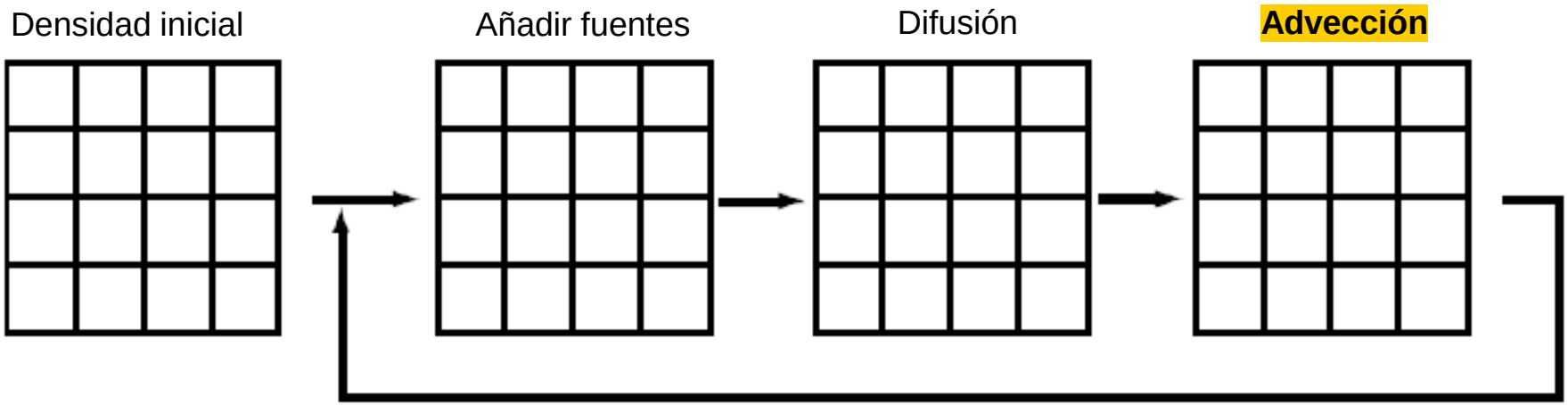
- Método es estable.
- Es necesario resolver este sistema rápido:

  Métodos iterativos (Jacobi, **Gauss-Seidel**, SOR)

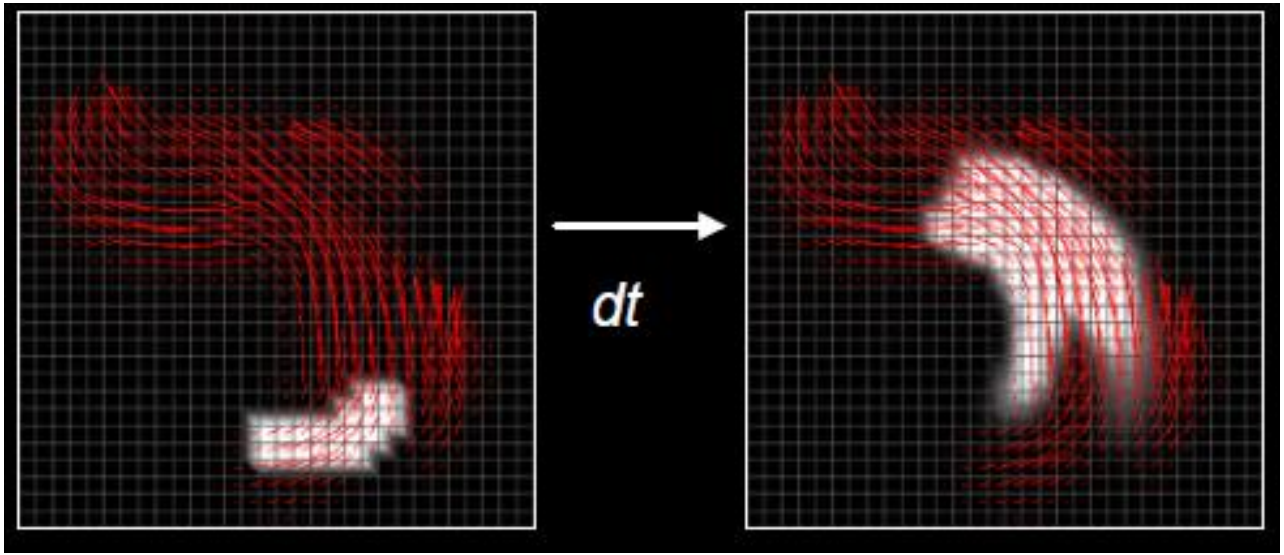
  Gradiente conjugado

  ....

Algoritmo : advección



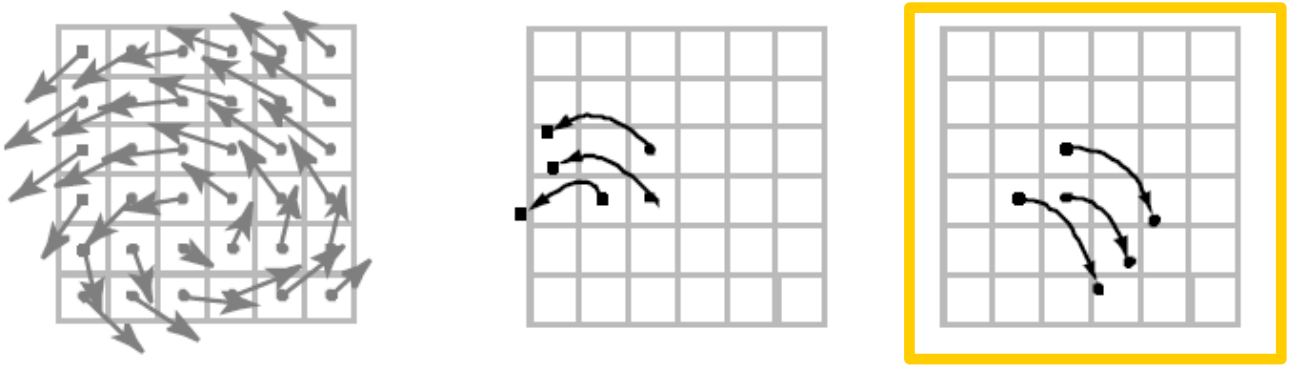
Algoritmo : advección



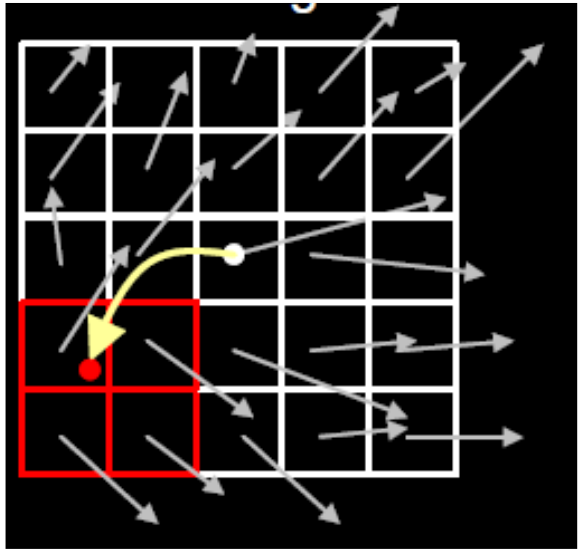
Dada un campo vectorial de velocidades, como se mueven las densidades.

Algoritmo : advección

- Queremos método estable: ***backwards.***
- Replicar método de difusión: muy costoso (las ecuaciones lineales dependen ahora de la velocidad).
- Idea: ver a donde se mueve el centro de cada casilla.



Algoritmo : advección



- ¿Qué valor de densidad transportamos?

- Cogemos las cuatro celdas más cercanas e interpolamos linealmente.